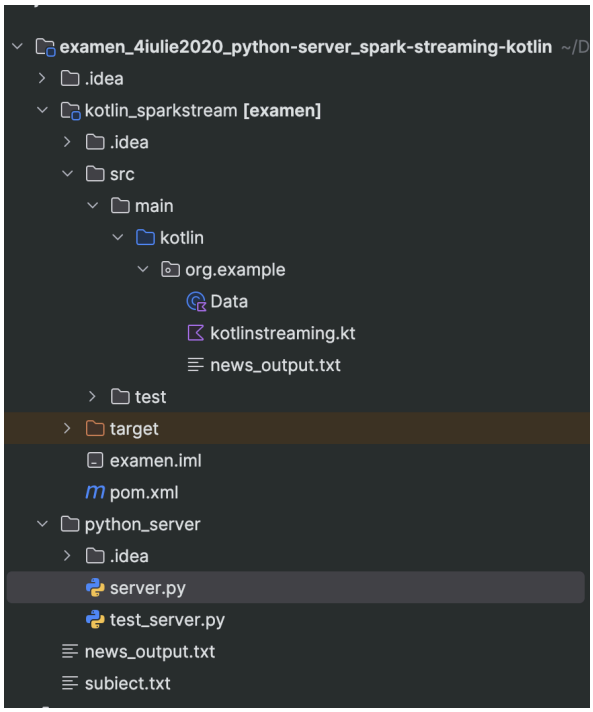
Să se creeze în Kotlin un server TCP care să facă o cerere (khttp) pentru simbolurile companiilor cu acțiuni (vezi documentația: https://finnhub.io/docs/api#stock-symbols) și folosind aceste simboluri precum și API-ul de știri despre o companie (https://finnhub.io/docs/api#company-news) să trimită știrile din ziua precedentă prin socket, câte o știre o dată la 3 secunde (pentru serializare/deserializare se pot utiliza funcțiile loads și dumps din
modulul json din Python, iar în Kotlin se poate utiliza kotlinx.serialization https://github.com/Kotlin/kotlinx.serialization). De asemenea, se va implementa în Python un flux de date direct (direct stream) utilizând framework-ul Apache Spark (PySpark), care să preia datele de la
serverul TCP și să le prelucreze astfel:
- se vor filtra acele știri care folosesc o imagine PNG;
- se vor filtra acele știri al căror URL nu depășește 80 de caractere;
- pentru fiecare RDD în parte, se va afișa URL-ul corespunzător
știrii, data și titlul.
Există o limitare de 60 de apeluri / minut la fiecare cheie API. Se poate crea un cont pe platforma finnhub.io (https://finnhub.io/register), sau se poate utiliza următoarea cheie (token) API: brmrfu7rh5rcss140ogg



NOTITE:
1. Pornesti server-ul si dupa rulezi kotlinstreaming.kt
2. Posibil sa ii dea eroare ca nu are pachetul requests(stanga jos la pachete il cauti si il instalezi)
3. Trebuie sa ai un JDK sub 17

Data.kt

```
package org.example

import kotlinx.serialization.Serializable

@Serializable
data class Data(
    val headline: String = "",
    val url: String = "",
    val image: String = "",
    val datetime: Long = 0L,
)
```

kotlinstreaming.kt

```
@file:Suppress("ktlint:standard:filename")

package org.example

import kotlinx.serialization.decodeFromString
import kotlinx.serialization.json.Json
import org.apache.spark.SparkConf
import org.apache.spark.streaming.Durations
import org.apache.spark.streaming.api.java.JavaStreamingContext
import java.io.File
import java.io.FileWriter
import java.time.Instant
import java.time.ZoneId
import java.time.format.DateTimeFormatter

fun main() {
    val streamIP = "localhost"
    val streamSocket = 1502
    val sparkConfig =
        SparkConf()
            .setMaster("local[*]")
            .setAppName("kotlin_streaming")
            .set("spark.executor.cores", "*")

    val streamingContext = JavaStreamingContext(sparkConfig,
Durations.seconds(3))

    val data = streamingContext.socketTextStream(streamIP, streamSocket)
    val dataJSON =
        data
            .map {
```

```
                Json { ignoreUnknownKeys = true }.decodeFromString<Data>(it)
            }

    dataJSON.foreachRDD { dataRDD ->
        val file = File("news_output.txt")
        val writer = FileWriter(file, true)

        dataRDD.collect().forEach { news ->
            val formattedDate =
                Instant
                    .ofEpochSecond(news.datetime)
                    .atZone(ZoneId.systemDefault())
                    .format(DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm"))

            writer.write("Titlu: ${news.headline}\n")
            writer.write("Data: $formattedDate\n")
            writer.write("URL: ${news.url}\n")
            writer.write("---------------\n")
        }
        writer.flush()
        writer.close()
    }

    streamingContext.start()
    streamingContext.awaitTermination()
    streamingContext.stop()
}
```

Pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/maven-v4_0_0.xsd">

    <modelVersion>4.0.0</modelVersion>

    <groupId>org.example</groupId>
    <artifactId>examen</artifactId>
    <version>1.0.0</version>
    <packaging>jar</packaging>

    <name>org.example examen</name>

    <properties>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
        <kotlin.version>1.4.0</kotlin.version>
```

```
    <kotlin.code.style>official</kotlin.code.style>
    <junit.version>4.12</junit.version>
    <serialization.version>1.0-M1-1.4.0-rc</serialization.version>
</properties>

<dependencies>
    <dependency>
        <groupId>org.jetbrains.kotlin</groupId>
        <artifactId>kotlin-maven-plugin</artifactId>
        <version>${kotlin.version}</version>
        <scope>provided</scope>
    </dependency>
    <!-- Apache RDD. -->
    <dependency>
        <groupId>org.apache.spark</groupId>
        <artifactId>spark-core_2.12</artifactId>
        <version>2.4.5</version>
    </dependency>
    <!-- Apache spark streaming. -->
    <dependency>
        <groupId>org.apache.spark</groupId>
        <artifactId>spark-streaming_2.12</artifactId>
        <version>2.4.5</version>
    </dependency>
    <!-- Kotlinx serialization JSON. -->
    <dependency>
        <groupId>org.jetbrains.kotlinx</groupId>
        <artifactId>kotlinx-serialization-runtime</artifactId>
        <version>${serialization.version}</version>
    </dependency>
    <dependency>
        <groupId>org.jetbrains.kotlin</groupId>
        <artifactId>kotlin-stdlib</artifactId>
        <version>${kotlin.version}</version>
    </dependency>
    <dependency>
        <groupId>org.jetbrains.kotlin</groupId>
        <artifactId>kotlin-test-junit</artifactId>
        <version>${kotlin.version}</version>
        <scope>test</scope>
    </dependency>
    <dependency>
        <groupId>junit</groupId>
        <artifactId>junit</artifactId>
        <version>${junit.version}</version>
        <scope>test</scope>
    </dependency>
</dependencies>
```

```
    <build>
        <sourceDirectory>src/main/kotlin</sourceDirectory>
        <testSourceDirectory>src/test/kotlin</testSourceDirectory>

        <plugins>
            <plugin>
                <groupId>org.jetbrains.kotlin</groupId>
                <artifactId>kotlin-maven-plugin</artifactId>
                <version>${kotlin.version}</version>
                <executions>
                    <execution>
                        <id>compile</id>
                        <phase>compile</phase>
                        <goals>
                            <goal>compile</goal>
                        </goals>
                    </execution>
                </executions>
                <configuration>
                    <compilerPlugins>
                        <plugin>kotlinx-serialization</plugin>
                    </compilerPlugins>
                </configuration>
                <dependencies>
                    <dependency>
                        <groupId>org.jetbrains.kotlin</groupId>
                        <artifactId>kotlin-maven-serialization</artifactId>
                        <version>1.8.22</version>
                    </dependency>
                </dependencies>
            </plugin>
        </plugins>
    </build>

</project>
```

server.py

```
# Python TCP Server - sends news data instead of profiles
import json
import socket
import time
import requests
from typing import List
from datetime import datetime, timedelta


class Api3rdParty:
```

```
    def __init__(self):
        self.__api_uri = "https://finnhub.io/api/v1/"
        self.__token = "brmrfu7rh5rcss140ogg"
        self.api_url_stock_symbols =
f"{self.__api_uri}stock/symbol?exchange=US&token={self.__token}"

    def getData(self) -> List[dict]:
        response = requests.get(self.api_url_stock_symbols)
        return response.json()

    def getSpecificData(self, symbol: str) -> List[dict]:
        today = datetime.now()
        yesterday = today - timedelta(days=1)
        from_date = yesterday.strftime("%Y-%m-%d")
        to_date = today.strftime("%Y-%m-%d")
        url =
f"{self.__api_uri}company-news?symbol={symbol}&from={from_date}&to={to_date}&to
ken={self.__token}"
        try:
            return requests.get(url).json()
        except:
            return []


class ApiNetworkService:
    def __init__(self, host: str, port: int):
        self.__socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        self.__socket.bind((host, port))
        self.__socket.listen(1)
        self.__client_socket, _ = self.__socket.accept()

    def sendJSON(self, json_data: str):
        self.__client_socket.sendall((json_data + "\n").encode())

    def close(self):
        self.__client_socket.close()
        self.__socket.close()


if __name__ == "__main__":
    party = Api3rdParty()
    net = ApiNetworkService("localhost", 1502)
    try:
        for company in party.getData():
            symbol = company.get("symbol")
            if not symbol:
                continue
            news_list = party.getSpecificData(symbol)
            for article in news_list:
```

```
            print(json.dumps(article))
            net.sendJSON(json.dumps(article))
            time.sleep(3)
    except Exception as e:
        print(f"Error: {e}")
    finally:
        net.close()
```